



MIDDLE EAST TECHNICAL UNIVERSITY

MMI714

Generative Models for Multimedia

Generative Adversarial Networks
(cont'd)

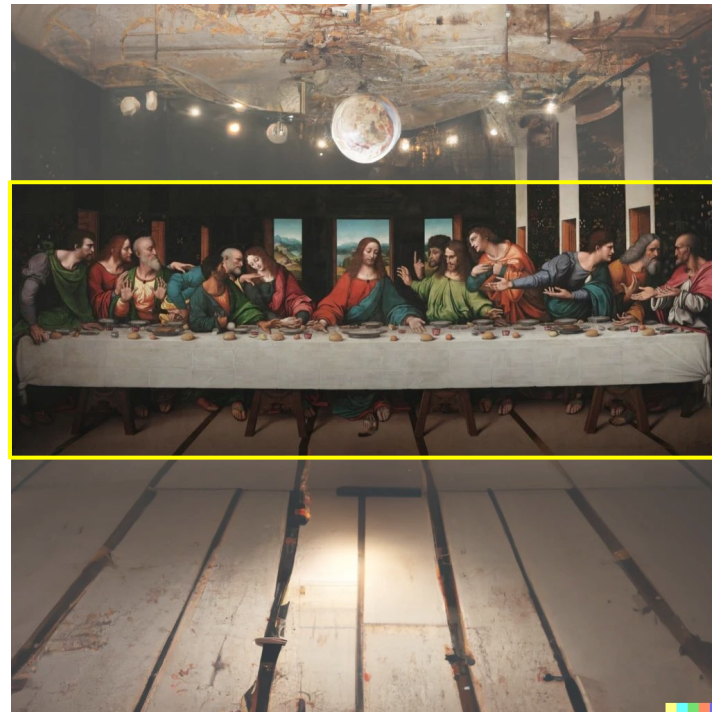
Welcome

This is MMI714

“Generative Models for Multimedia”

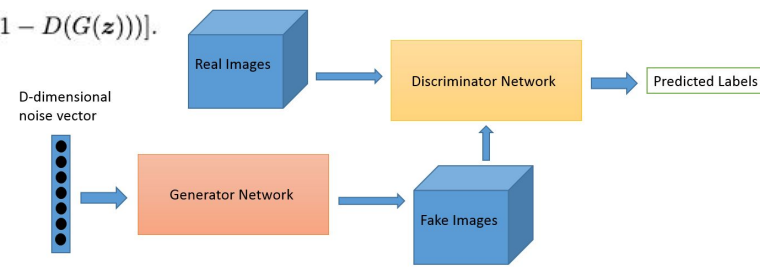
This week we will talk about:

Progressively Growing GANs,
Adaptive Instance Normalization,
StyleGAN, Path Length,
Disentanglement



$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

GANs (prev. week review)



- GANs consist of two neural networks, a generator, and a discriminator, engaged in an adversarial training process.
- The generator aims to create data that is indistinguishable from real data, while the discriminator role is to differentiate between real and generated data.
- DCGAN is a GAN with fully-convolutional generator. The generator mainly composes of convolution layers
- In cGANs the usual practise is some additional information being passed to the generator and/or discriminator networks.
- The Wasserstein loss serves as a metric promoting more stable and efficient learning by providing a smoother gradient for the generator.

Progressive Growing GANs

PROGRESSIVE GROWING OF GANs FOR IMPROVED QUALITY, STABILITY, AND VARIATION

Tero Karras
NVIDIA

Timo Aila
NVIDIA

Samuli Laine
NVIDIA

Jaakko Lehtinen
NVIDIA and Aalto University

{tkarras, taila, slaine, jlehtinen}@nvidia.com

- High-Resolution Generation!
- Progressively growing training approach!
 - Progressive Growing GAN is an extension to the GAN training process that allows for the stable training of generator models that can output larger high-quality images.
 - It involves starting with a very small image and incrementally adding blocks of layers that increase the output size of the generator model and the input size of the discriminator model until the desired image size is achieved.

ABSTRACT

We describe a new training methodology for generative adversarial networks. The key idea is to grow both the generator and discriminator progressively: starting from a low resolution, we add new layers that model increasingly fine details as training progresses. This both speeds the training up and greatly stabilizes it, allowing us to produce images of unprecedented quality, e.g., CELEBA images at 1024^2 . We also propose a simple way to increase the variation in generated images, and achieve a record inception score of 8.80 in unsupervised CIFAR10. Additionally, we describe several implementation details that are important for discouraging unhealthy competition between the generator and discriminator. Finally, we suggest a new metric for evaluating GAN results, both in terms of image quality and variation. As an additional contribution, we construct a higher-quality version of the CELEBA dataset.



Progressive Growing GANs

- A problem with GANs is that they are limited to small dataset sizes, often a few hundred pixels and often less than 100-pixel square images.
- *"GANs produce sharp images, albeit only in fairly small resolutions and with somewhat limited variation, and the training continues to be unstable despite recent progress."*
 - *"The generation of high-resolution images is difficult because higher resolution makes it easier to tell the generated images apart from training images."*

PROGRESSIVE GROWING OF GANs FOR IMPROVED QUALITY, STABILITY, AND VARIATION

Tero Karras
NVIDIA

Timo Aila
NVIDIA

Samuli Laine
NVIDIA

Jaakko Lehtinen
NVIDIA and Aalto University

{tkarras, taila, slaine, jlehtinen}@nvidia.com

ABSTRACT

We describe a new training methodology for generative adversarial networks. The key idea is to grow both the generator and discriminator progressively: starting from a low resolution, we add new layers that model increasingly fine details as training progresses. This both speeds the training up and greatly stabilizes it, allowing us to produce images of unprecedented quality, e.g., CELEBA images at 1024^2 . We also propose a simple way to increase the variation in generated images, and achieve a record inception score of 8.80 in unsupervised CIFAR10. Additionally, we describe several implementation details that are important for discouraging unhealthy competition between the generator and discriminator. Finally, we suggest a new metric for evaluating GAN results, both in terms of image quality and variation. As an additional contribution, we construct a higher-quality version of the CELEBA dataset.



Progressive Growing GANs

- *"Our key insight is that we can grow both the generator and discriminator progressively, starting from easier low-resolution images, and add new layers that introduce higher-resolution details as the training progresses."*
- *"Our primary contribution is a training methodology for GANs where we start with low-resolution images, and then progressively increase the resolution by adding layers to the networks."*
- *"This allows the training to first discover large-scale structure of the image distribution and then shift attention to increasingly finer scale detail, instead of having to learn all scales simultaneously."*

PROGRESSIVE GROWING OF GANs FOR IMPROVED QUALITY, STABILITY, AND VARIATION

Tero Karras
NVIDIA

Timo Aila
NVIDIA

Samuli Laine
NVIDIA

Jaakko Lehtinen
NVIDIA and Aalto University

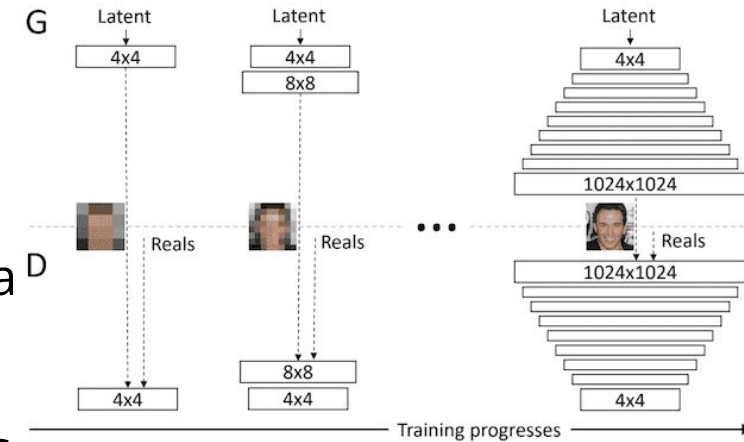
{tkarras, taila, slaine, jlehtinen}@nvidia.com

ABSTRACT

We describe a new training methodology for generative adversarial networks. The key idea is to grow both the generator and discriminator progressively: starting from a low resolution, we add new layers that model increasingly fine details as training progresses. This both speeds the training up and greatly stabilizes it, allowing us to produce images of unprecedented quality, e.g., CELEBA images at 1024^2 . We also propose a simple way to increase the variation in generated images, and achieve a record inception score of 8.80 in unsupervised CIFAR10. Additionally, we describe several implementation details that are important for discouraging unhealthy competition between the generator and discriminator. Finally, we suggest a new metric for evaluating GAN results, both in terms of image quality and variation. As an additional contribution, we construct a higher-quality version of the CELEBA dataset.

Progressive Growing GANs

- Progressive Growing GAN involves using a generator and discriminator model with the same general structure and starting with very small images, such as 4x4 pixels.
- During training, **new blocks of convolutional layers are systematically added to both the generator model and the discriminator models.**



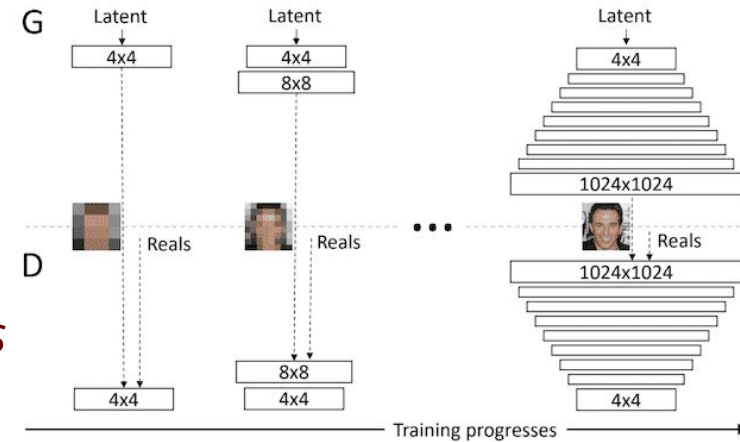
2 PROGRESSIVE GROWING OF GANs

Our primary contribution is a training methodology for GANs where we start with low-resolution images, and then progressively increase the resolution by adding layers to the networks as visualized in Figure 1. This incremental nature allows the training to first discover large-scale structure of the image distribution and then shift attention to increasingly finer scale detail, instead of having to learn all scales simultaneously.

We use generator and discriminator networks that are mirror images of each other and always grow in synchrony. All existing layers in both networks remain trainable throughout the training process. When new layers are added to the networks, we fade them in smoothly, as illustrated in Figure 2. This avoids sudden shocks to the already well-trained, smaller-resolution layers. Appendix A describes structure of the generator and discriminator in detail, along with other training parameters.

Progressive Growing GANs

- *"Early on, the generation of smaller images is substantially more stable because there is less class information and fewer modes."*
- *"By increasing the resolution little by little we are continuously asking a much simpler question compared to the end goal of discovering a mapping from latent (512) vectors to 1024^2 "*
- *"This approach has conceptual similarity to recent work by Chen&Koltun (2017). In practice it stabilizes the training sufficiently for us to reliably synthesize megapixel-scale images"*



Progressive Growing GANs

- *"Early on, the generation of smaller images is substantially more stable because there is less class information and fewer modes."*
- *"By increasing the resolution little by little, it is a much simpler question compared to the one-to-one mapping from latent (512) vectors to 1 megapixel images."*
- *"This approach has conceptual similarity to recent work by Chen&Koltun (2017). In practice it stabilizes the training sufficiently for us to reliably synthesize megapixel-scale images"*



Figure 1. Given a pixelwise semantic layout, the presented model synthesizes an image that conforms to this layout. (a) Semantic layouts from the Cityscapes dataset of urban scenes; semantic classes are coded by color. (b) Images synthesized by our model for these layouts. The layouts shown here and throughout the paper are from the validation set and depict scenes from new cities that were never seen during training. Best viewed on the screen.

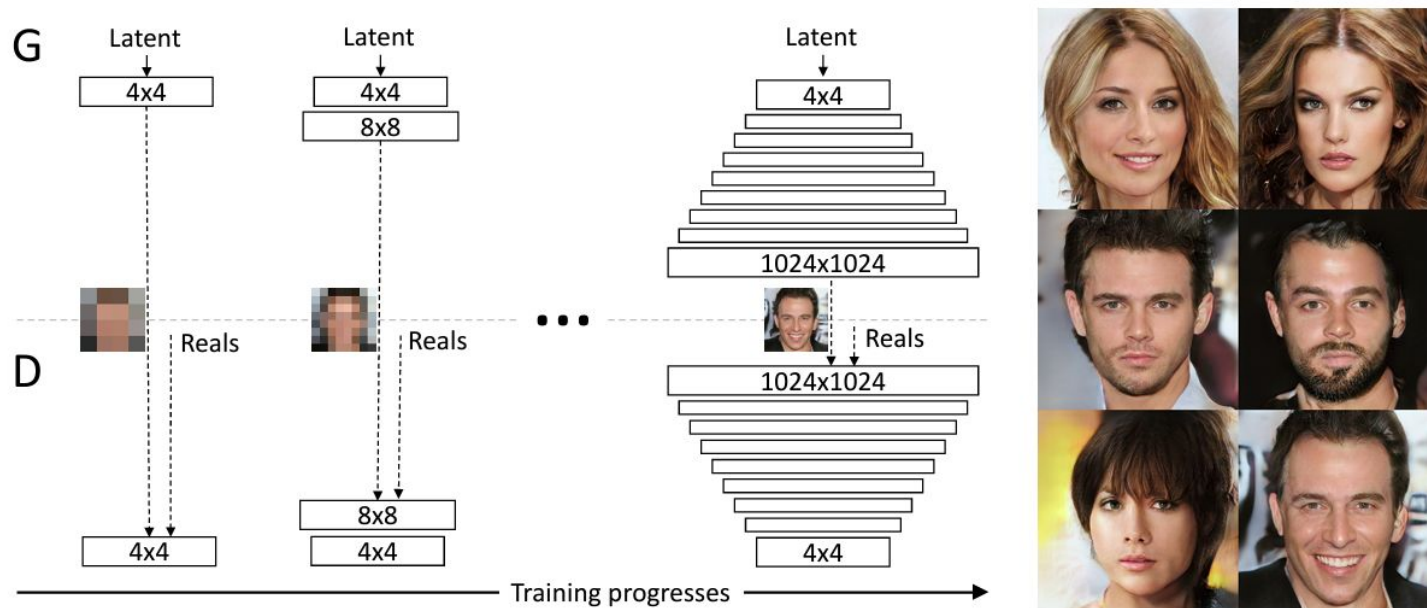
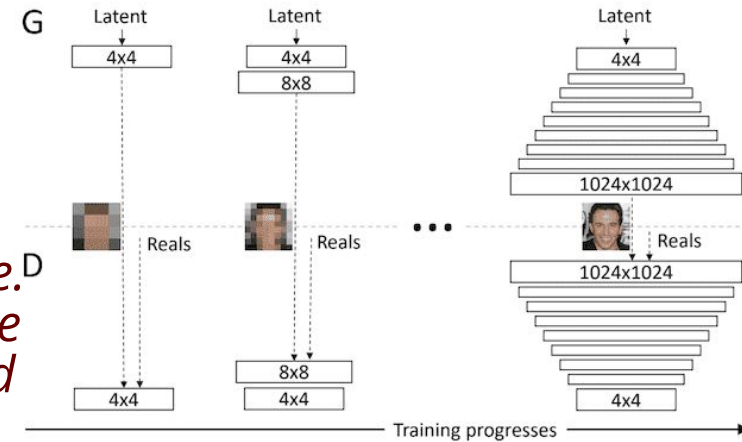


Figure 1: Our training starts with both the generator (G) and discriminator (D) having a low spatial resolution of 4×4 pixels. As the training advances, we incrementally add layers to G and D, thus increasing the spatial resolution of the generated images. All existing layers remain trainable throughout the process. Here $N \times N$ refers to convolutional layers operating on $N \times N$ spatial resolution. This allows stable synthesis in high resolutions and also speeds up training considerably. On the right we show six example images generated using progressive growing at 1024×1024 .

Progressive Growing GANs

- “Another benefit is the reduced training time. With progressively growing GANs most of the iterations are done at lower resolutions, and comparable result quality is often obtained up to 2–6 times faster, depending on the final output resolution.”*
- “By increasing the resolution little by little we are continuously asking a much simpler question compared to the end goal of discovering a mapping from latent (512) vectors to 1024^2 ”*
- “We primarily use the **improved Wasserstein loss**, but also experiment with least-squares loss.”*



Progressive Growing GANs

- *“The paper proposes a new penalty term called WGAN-GP (Wasserstein GAN with Gradient Penalty) that replaces weight clipping in WGAN.”*
- *“The WGAN-GP method improves the convergence properties of the value function being optimized by GANs and provides stable training of varied GAN architectures, performance improvements over weight clipping, high-quality image generation, and a character-level GAN language model”.*

Ishaan Gulrajani¹, Faruk Ahmed¹, Martin Arjovsky², Vincent Dumoulin¹, Aaron Courville^{1,3}

¹ Montreal Institute for Learning Algorithms

² Courant Institute of Mathematical Sciences

³ CIFAR Fellow

igul222@gmail.com

{faruk.ahmed, vincent.dumoulin, aaron.courville}@umontreal.ca
ma4371@nyu.edu

Abstract

Generative Adversarial Networks (GANs) are powerful generative models, but suffer from training instability. The recently proposed Wasserstein GAN (WGAN) makes progress toward stable training of GANs, but sometimes can still generate only poor samples or fail to converge. We find that these problems are often due to the use of weight clipping in WGAN to enforce a Lipschitz constraint on the critic, which can lead to undesired behavior. We propose an alternative to clipping weights: penalize the norm of gradient of the critic with respect to its input. Our proposed method performs better than standard WGAN and enables stable training of a wide variety of GAN architectures with almost no hyperparameter tuning, including 101-layer ResNets and language models with continuous generators. We also achieve high quality generations on CIFAR-10 and LSUN bedrooms. [†]

ProGAN results

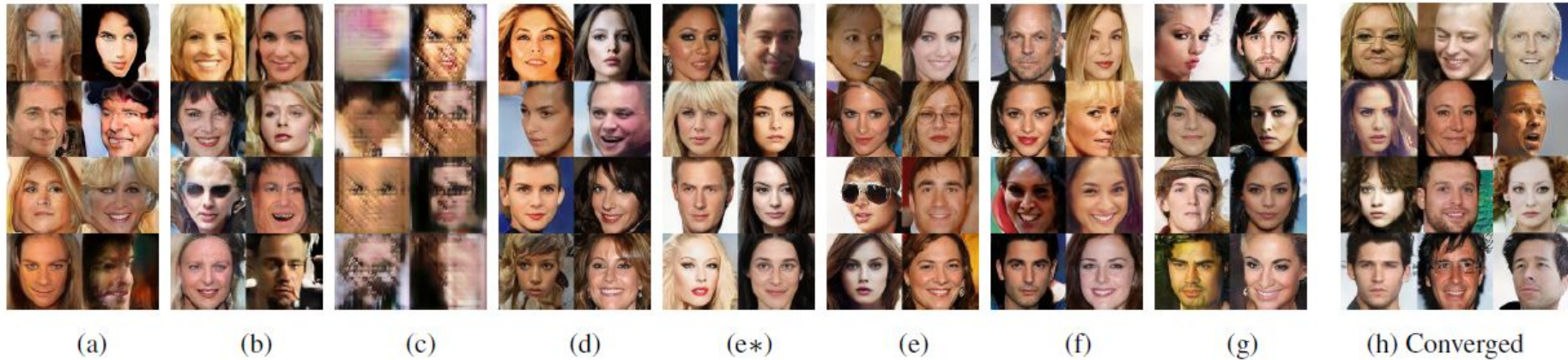


Figure 3: (a) – (g) CELEBA examples corresponding to rows in Table 1. These are intentionally non-converged. (h) Our converged result. Notice that some images show aliasing and some are not sharp – this is a flaw of the dataset, which the model learns to replicate faithfully.

Evaluation

- First, to evaluate the importance of individual contributions in terms of statistical similarity, they use:
 - the sliced Wasserstein distance (SWD)
- SWD: is a version of Wasserstein distance, which is a measure of distance between two distributions.
- In the context of generative models like GANs, the "direction" in Sliced Wasserstein Distance (SWD) typically refers to a random vector in the latent space.

$$\text{SWD}(P, Q) = \frac{1}{n} \sum_{i=1}^n W(P_{u_i}, Q_{u_i})$$

where n is the number of random directions sampled, and u_i represents the i -th randomly chosen direction.

Evaluation

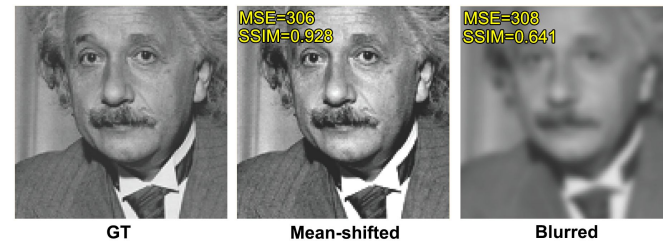
- First, to evaluate the importance of individual contributions in terms of statistical similarity, they **also** use:
 - multi-scale structural similarity (MS-SSIM)
- SSIM: is a perceptual metric. It measures the similarity in structure between real and generated images, taking into account luminance, contrast, and structure.
- MS-SSIM: SSIM applied on a pyramid.

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

with:

- μ_x the average of x ;
- μ_y the average of y ;
- σ_x^2 the variance of x ;
- σ_y^2 the variance of y ;
- σ_{xy} the covariance of x and y ;
- $c_1 = (k_1 L)^2$, $c_2 = (k_2 L)^2$ two variables to stabilize the division with weak denominator;
- L the dynamic range of the pixel-values (typically this is $2^{\text{\#bits per pixel}} - 1$);
- $k_1 = 0.01$ and $k_2 = 0.03$ by default.

Evaluation



- First, to evaluate the importance of individual contributions in terms of statistical similarity, they **also** use:
 - multi-scale structural similarity (MS-SSIM)
- SSIM: is a perceptual metric. It measures the similarity in structure between real and generated images, taking into account luminance, contrast, and structure.
- MS-SSIM: SSIM applied on a pyramid.

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)}$$

with:

- μ_x the average of x ;
- μ_y the average of y ;
- σ_x^2 the variance of x ;
- σ_y^2 the variance of y ;
- σ_{xy} the covariance of x and y ;
- $c_1 = (k_1 L)^2$, $c_2 = (k_2 L)^2$ two variables to stabilize the division with weak denominator;
- L the dynamic range of the pixel-values (typically this is $2^{\text{\#bits per pixel}} - 1$);
- $k_1 = 0.01$ and $k_2 = 0.03$ by default.

$$IS(G) = \exp \left(\mathbb{E}_{\mathbf{x} \sim p_g} D_{KL}(p(y|\mathbf{x}) \parallel p(y)) \right),$$

Evaluation

- Second, to evaluate the evaluate the **quality & diversity** of the generated (multiple) images, they use:
 - Inception Score (IS)
- IS: is a deep-classification-based metric. It involves classifying generated images using a pre-trained InceptionV3 model.
- The score is a combination of the model's confidence in its predictions (high for realistic images) and the diversity of predictions across different classes (high for diverse images).
- (Very simply) KL between real/generated class distributions

$$\text{FID} = \|\mu_{\text{real}} - \mu_{\text{gen}}\|^2 + \text{Tr}(\Sigma_{\text{real}} + \Sigma_{\text{gen}} - 2(\Sigma_{\text{real}}\Sigma_{\text{gen}})^{1/2})$$

where μ is the mean and Σ is the covariance matrix.

Evaluation

- Second, to evaluate the evaluate the **quality & diversity** of the generated (multiple) images, they **also** use:
 - Fréchet Inception Distance (FID):
- FID: is a deep-feature-based metric. It typically uses activations from a specific layer of a pre-trained InceptionV3 model.
- FID computes the “Fréchet” distance between the “feature” distributions.
- The Fréchet distance is a metric in a space that considers both the mean and covariance of two distributions.

$$\text{FID} = \|\mu_{\text{real}} - \mu_{\text{gen}}\|^2 + \text{Tr}(\Sigma_{\text{real}} + \Sigma_{\text{gen}} - 2(\Sigma_{\text{real}}\Sigma_{\text{gen}})^{1/2})$$

where μ is the mean and Σ is the covariance matrix.

Evaluation

- But InceptionV3 is trained on 1000 class ImageNet! What if I am using, say, sound? (or earthquake accelerometer records, like we do)
- Then you retrain the inceptionV3 (or some other classifier) on your modality to get a specific FID (or IS) score.
- For example:



CVPR2023

This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

Physics-Driven Diffusion Models for Impact Sound Synthesis from Videos

Kun Su^{1*} Kaizhi Qian²
¹University of Washington

Eli Shlizerman¹ Antonio Torralba³
²MIT-IBM Watson AI Lab ³MIT

Chuang Gan^{2,4}
⁴UMass Amherst

4.4. Evaluation Metrics

We use four different metrics to automatically assess both the fidelity and relevance of the generated samples. For automatic evaluation purpose, we train an impact sound object material classifier using the labels in Great Hits Dataset. The classifier is a ResNet-50 convolutional-based neural network and we use the spectrogram as input to train the classifier.

- **Fréchet Inception Distance (FID)** is used for evaluating the quality of generated impact sound spectrograms. The FID score evaluates the distance between the distribution of synthesized spectrograms and the spectrograms in the test set. To build the distribution, we extract the features before the impact sound classification layer.
- **Kernel Inception Distance (KID)** is calculated via maximum mean discrepancy (MMD). Again, we extract features from synthesized and real impact sounds. The MMD is calculated over a number of subsets to both get the mean and standard deviation of KID.
- **KL Divergence** is used to individually compute the distance between output distributions of synthesized and ground truth features since FID and KID mainly rely on the distribution of a collection of samples.
- **Recognition accuracy** is used to evaluate if the quality of generated impact sound samples can fool the classifier.



Evaluation

- What if there is no off-the-shelf classifier for your modality?
 - Wait! What! Is there such data? It is 2025 already!
- Yes. There is such data. There are tens of DL studies on strong motion records, but apparently none of them is an AlexNet for EQ engineering.
- Then you would not be able to use an FID score to evaluate your generative model (which is our case).
- But maybe, you may use an FID score **to evaluate the success of the (unsupervised or supervised) learned features for a task**, like regional discrimination (like two records from different stations having a larger FI distance, compared to the self distance a station has)
- Can this could be a measure for how good deep features you construct!

StyleGAN

- Same group (NVIDIA) proposed the StyleGAN about a year later in CVPR.
- StyleGAN follows the Progressive GAN in using a progressively growing training regime.
- Its contributions are
 - using an alternative generator architecture for generative adversarial networks,
 - borrowing from style transfer literature; in particular, the use of **adaptive instance normalization (AdaIN)**.

Tero Karras
NVIDIA

tkarras@nvidia.com

Samuli Laine
NVIDIA

slaine@nvidia.com

Timo Aila
NVIDIA

taila@nvidia.com

Abstract

We propose an alternative generator architecture for generative adversarial networks, borrowing from style transfer literature. The new architecture leads to an automatically learned, unsupervised separation of high-level attributes (e.g., pose and identity when trained on human faces) and stochastic variation in the generated images (e.g., freckles, hair), and it enables intuitive, scale-specific control of the synthesis. The new generator improves the state-of-the-art in terms of traditional distribution quality metrics, leads to demonstrably better interpolation properties, and also better disentangles the latent factors of variation. To quantify interpolation quality and disentanglement, we propose two new, automated methods that are applicable to any generator architecture. Finally, we introduce a new, highly varied and high-quality dataset of human faces.

(e.g., pose, identity) from stochastic variation (e.g., freckles, hair) in the generated images, and enables intuitive scale-specific mixing and interpolation operations. We do not modify the discriminator or the loss function in any way, and our work is thus orthogonal to the ongoing discussion about GAN loss functions, regularization, and hyperparameters [24, 45, 5, 40, 44, 36].

Our generator embeds the input latent code into an intermediate latent space, which has a profound effect on how the factors of variation are represented in the network. The input latent space must follow the probability density of the training data, and we argue that this leads to some degree of unavoidable entanglement. Our intermediate latent space is free from that restriction and is therefore allowed to be disentangled. As previous methods for estimating the degree of latent space disentanglement are not directly applicable in our case, we propose two new automated metrics—

3v3 [cs.NE] 29 Mar 2019

Adaptive Instance Normalization

- (very simply) AdaIN is a normalization method that aligns the mean and variance of the content features with those of the style features.
- In AdaIN, we receive a content input and a style input, and we simply align the channel-wise mean and variance of the target to match those of the style input.
- AdaIN has no learnable affine parameters (like BN).

$$\text{AdaIN}(x, y) = \sigma(y) \left(\frac{x - \mu(x)}{\sigma(x)} \right) + \mu(y)$$

Abstract

Gatys et al. recently introduced a neural algorithm that renders a content image in the style of another image, achieving so-called style transfer. However, their framework requires a slow iterative optimization process, which limits its practical application. Fast approximations with feed-forward neural networks have been proposed to speed up neural style transfer. Unfortunately, the speed improvement comes at a cost: the network is usually tied to a fixed set of styles and cannot adapt to arbitrary new styles. In this paper, we present a simple yet effective approach that for the first time enables arbitrary style transfer in real-time. At the heart of our method is a novel adaptive instance normalization (AdaIN) layer that aligns the mean and variance of the content features with those of the style features. Our method

dilemma. Our approach can transfer arbitrary new styles in real-time, combining the flexibility of the optimization-based framework [16] and the speed similar to the fastest feed-forward approaches [24, 52]. Our method is inspired by the instance normalization (IN) [52, 11] layer, which is surprisingly effective in feed-forward style transfer. To explain the success of instance normalization, we propose a new interpretation that instance normalization performs style normalization by normalizing feature statistics, which have been found to carry the style information of an image [16, 30, 33]. Motivated by our interpretation, we introduce a simple extension to IN, namely *adaptive instance normalization* (AdaIN). Given a content input and a style input, AdaIN simply adjusts the mean and variance of the content input to match those of the style input. Through experiments, we find AdaIN effectively combines the con-

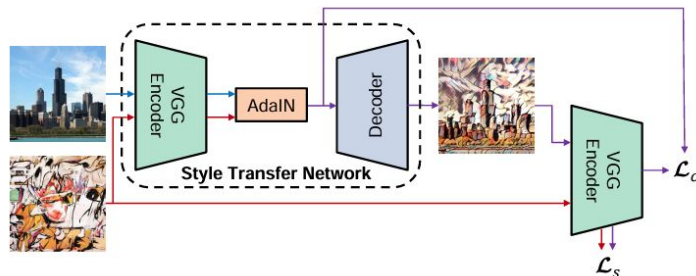


Figure 2. An overview of our style transfer algorithm. We use the first few layers of a fixed VGG-19 network to encode the content and style images. An AdaIN layer is used to perform style transfer in the feature space. A decoder is learned to invert the AdaIN output to the image spaces. We use the same VGG encoder to compute a content loss $\mathcal{L}_c$ (Equ. 12) and a style loss $\mathcal{L}_s$ (Equ. 13).

Remember the app Prisma?



for more
on AdaIN
check [1](#) [2](#)

Adaptive Instance Normalization

- Style Loss ($\mathcal{L}_s$): measures the difference between the Gram matrices of the stylized image and the style image.
- The Gram matrix is a measure of the **correlations** between the different feature maps in a given layer of the network.

Abstract

Gatys et al. recently introduced a neural algorithm that renders a content image in the style of another image, achieving so-called style transfer. However, their framework requires a slow iterative optimization process, which limits its practical application. Fast approximations with feed-forward neural networks have been proposed to speed up neural style transfer. Unfortunately, the speed improvement comes at a cost: the network is usually tied to a fixed set of styles and cannot adapt to arbitrary new styles. In this paper, we present a simple yet effective approach that for the first time enables arbitrary style transfer in real-time. At the heart of our method is a novel adaptive instance normalization (AdaIN) layer that aligns the mean and variance of the content features with those of the style features. Our method

dilemma. Our approach can transfer arbitrary new styles in real-time, combining the flexibility of the optimization-based framework [16] and the speed similar to the fastest feed-forward approaches [24, 52]. Our method is inspired by the instance normalization (IN) [52, 11] layer, which is surprisingly effective in feed-forward style transfer. To explain the success of instance normalization, we propose a new interpretation that instance normalization performs style normalization by normalizing feature statistics, which have been found to carry the style information of an image [16, 30, 33]. Motivated by our interpretation, we introduce a simple extension to IN, namely *adaptive instance normalization* (AdaIN). Given a content input and a style input, AdaIN simply adjusts the mean and variance of the content input to match those of the style input. Through experiments, we find AdaIN effectively combines the con-

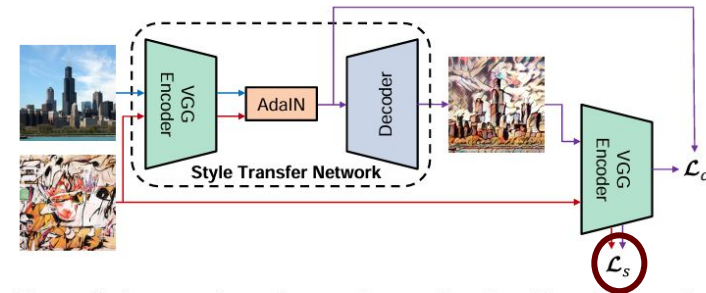


Figure 2. An overview of our style transfer algorithm. We use the first few layers of a fixed VGG-19 network to encode the content and style images. An AdaIN layer is used to perform style transfer in the feature space. A decoder is learned to invert the AdaIN output to the image spaces. We use the same VGG encoder to compute a content loss $\mathcal{L}_c$ (Equ. 12) and a style loss $\mathcal{L}_s$ (Equ. 13).

Remember the app Prisma?



for more
on AdaIN
check 1 2

Adaptive Instance Normalization

- the Gram matrix of the style image is computed using the feature maps of the style image, while the Gram matrix of the stylized image is computed using the feature maps of the stylized image.
- $\mathcal{L}_s$ measures the difference between these two Gram matrices, which captures the differences in the correlations between the feature maps of the style image and the stylized image.

Abstract

Gatys et al. recently introduced a neural algorithm that renders a content image in the style of another image, achieving so-called style transfer. However, their framework requires a slow iterative optimization process, which limits its practical application. Fast approximations with feed-forward neural networks have been proposed to speed up neural style transfer. Unfortunately, the speed improvement comes at a cost: the network is usually tied to a fixed set of styles and cannot adapt to arbitrary new styles. In this paper, we present a simple yet effective approach that for the first time enables arbitrary style transfer in real-time. At the heart of our method is a novel adaptive instance normalization (AdaIN) layer that aligns the mean and variance of the content features with those of the style features. Our method

dilemma. Our approach can transfer arbitrary new styles in real-time, combining the flexibility of the optimization-based framework [16] and the speed similar to the fastest feed-forward approaches [24, 52]. Our method is inspired by the instance normalization (IN) [52, 11] layer, which is surprisingly effective in feed-forward style transfer. To explain the success of instance normalization, we propose a new interpretation that instance normalization performs style normalization by normalizing feature statistics, which have been found to carry the style information of an image [16, 30, 33]. Motivated by our interpretation, we introduce a simple extension to IN, namely *adaptive instance normalization* (AdaIN). Given a content input and a style input, AdaIN simply adjusts the mean and variance of the content input to match those of the style input. Through experiments, we find AdaIN effectively combines the con-

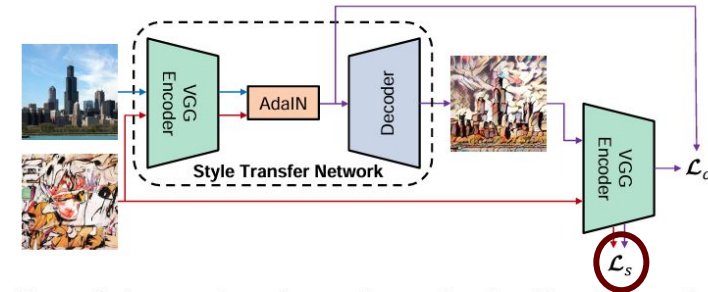


Figure 2. An overview of our style transfer algorithm. We use the first few layers of a fixed VGG-19 network to encode the content and style images. An AdaIN layer is used to perform style transfer in the feature space. A decoder is learned to invert the AdaIN output to the image spaces. We use the same VGG encoder to compute a content loss $\mathcal{L}_c$ (Equ. 12) and a style loss $\mathcal{L}_s$ (Equ. 13).

Remember the app Prisma?



for more
on AdaIN
check [1](#), [2](#)

Adaptive Instance Normalization

- Content Loss ($\mathcal{L}_c$): is the Euclidean distance between the target features and the features of the output image.
- The target features are obtained by passing the content image through the encoder network, while the features of the output image are obtained by passing the stylized image through the encoder network.

Abstract

Gatys et al. recently introduced a neural algorithm that renders a content image in the style of another image, achieving so-called style transfer. However, their framework requires a slow iterative optimization process, which limits its practical application. Fast approximations with feed-forward neural networks have been proposed to speed up neural style transfer. Unfortunately, the speed improvement comes at a cost: the network is usually tied to a fixed set of styles and cannot adapt to arbitrary new styles. In this paper, we present a simple yet effective approach that for the first time enables arbitrary style transfer in real-time. At the heart of our method is a novel adaptive instance normalization (AdaIN) layer that aligns the mean and variance of the content features with those of the style features. Our method

dilemma. Our approach can transfer arbitrary new styles in real-time, combining the flexibility of the optimization-based framework [16] and the speed similar to the fastest feed-forward approaches [24, 52]. Our method is inspired by the instance normalization (IN) [52, 11] layer, which is surprisingly effective in feed-forward style transfer. To explain the success of instance normalization, we propose a new interpretation that instance normalization performs style normalization by normalizing feature statistics, which have been found to carry the style information of an image [16, 30, 33]. Motivated by our interpretation, we introduce a simple extension to IN, namely adaptive instance normalization (AdaIN). Given a content input and a style input, AdaIN simply adjusts the mean and variance of the content input to match those of the style input. Through experiments, we find AdaIN effectively combines the con-

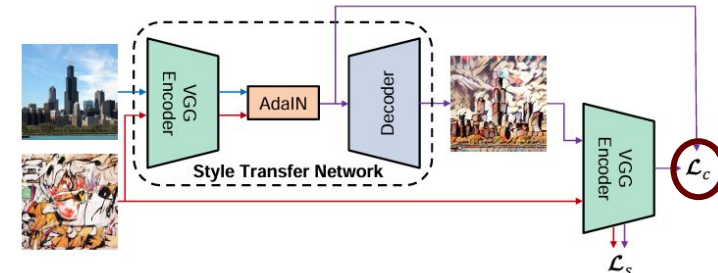


Figure 2. An overview of our style transfer algorithm. We use the first few layers of a fixed VGG-19 network to encode the content and style images. An AdaIN layer is used to perform style transfer in the feature space. A decoder is learned to invert the AdaIN output to the image spaces. We use the same VGG encoder to compute a content loss $\mathcal{L}_c$ (Equ. 12) and a style loss $\mathcal{L}_s$ (Equ. 13).

Remember the app Prisma?



for more
on AdaIN
check 1.2

Adaptive Instance Normalization

- Only the decoder network is trained!
- The encoder networks for the content and style images are pre-trained on a large dataset, such as ImageNet, and are fixed during training.

Abstract

Gatys et al. recently introduced a neural algorithm that renders a content image in the style of another image, achieving so-called style transfer. However, their framework requires a slow iterative optimization process, which limits its practical application. Fast approximations with feed-forward neural networks have been proposed to speed up neural style transfer. Unfortunately, the speed improvement comes at a cost: the network is usually tied to a fixed set of styles and cannot adapt to arbitrary new styles. In this paper, we present a simple yet effective approach that for the first time enables arbitrary style transfer in real-time. At the heart of our method is a novel adaptive instance normalization (AdaIN) layer that aligns the mean and variance of the content features with those of the style features. Our method

dilemma. Our approach can transfer arbitrary new styles in real-time, combining the flexibility of the optimization-based framework [16] and the speed similar to the fastest feed-forward approaches [24, 52]. Our method is inspired by the instance normalization (IN) [52, 11] layer, which is surprisingly effective in feed-forward style transfer. To explain the success of instance normalization, we propose a new interpretation that instance normalization performs style normalization by normalizing feature statistics, which have been found to carry the style information of an image [16, 30, 33]. Motivated by our interpretation, we introduce a simple extension to IN, namely adaptive instance normalization (AdaIN). Given a content input and a style input, AdaIN simply adjusts the mean and variance of the content input to match those of the style input. Through experiments, we find AdaIN effectively combines the con-

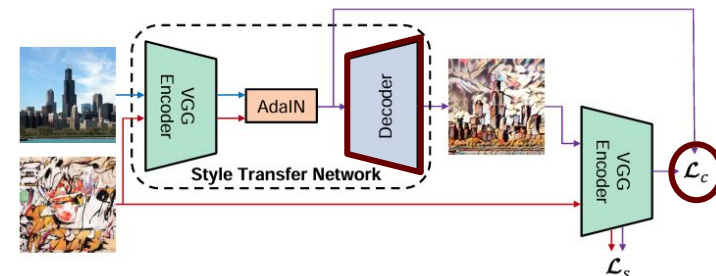


Figure 2. An overview of our style transfer algorithm. We use the first few layers of a fixed VGG-19 network to encode the content and style images. An AdaIN layer is used to perform style transfer in the feature space. A decoder is learned to invert the AdaIN output to the image spaces. We use the same VGG encoder to compute a content loss $\mathcal{L}_c$ (Equ. 12) and a style loss $\mathcal{L}_s$ (Equ. 13).

Remember the app Prisma?



for more
on AdaIN
check [1](#), [2](#)

StyleGAN

- Same group (NVIDIA) proposed the StyleGAN about a year later in CVPR.
- StyleGAN follows the Progressive GAN in using a progressively growing training regime.
- Its contributions are
 - using an alternative generator architecture for generative adversarial networks,
 - borrowing from style transfer literature; in particular, the use of **adaptive instance normalization (AdaIN). HOW?**

Tero Karras
NVIDIA

tkarras@nvidia.com

Samuli Laine
NVIDIA

slaine@nvidia.com

Timo Aila
NVIDIA

taila@nvidia.com

Abstract

We propose an alternative generator architecture for generative adversarial networks, borrowing from style transfer literature. The new architecture leads to an automatically learned, unsupervised separation of high-level attributes (e.g., pose and identity when trained on human faces) and stochastic variation in the generated images (e.g., freckles, hair), and it enables intuitive, scale-specific control of the synthesis. The new generator improves the state-of-the-art in terms of traditional distribution quality metrics, leads to demonstrably better interpolation properties, and also better disentangles the latent factors of variation. To quantify interpolation quality and disentanglement, we propose two new, automated methods that are applicable to any generator architecture. Finally, we introduce a new, highly varied and high-quality dataset of human faces.

(e.g., pose, identity) from stochastic variation (e.g., freckles, hair) in the generated images, and enables intuitive scale-specific mixing and interpolation operations. We do not modify the discriminator or the loss function in any way, and our work is thus orthogonal to the ongoing discussion about GAN loss functions, regularization, and hyperparameters [24, 45, 5, 40, 44, 36].

Our generator embeds the input latent code into an intermediate latent space, which has a profound effect on how the factors of variation are represented in the network. The input latent space must follow the probability density of the training data, and we argue that this leads to some degree of unavoidable entanglement. Our intermediate latent space is free from that restriction and is therefore allowed to be disentangled. As previous methods for estimating the degree of latent space disentanglement are not directly applicable in our case, we propose two new automated metrics—

3v3 [cs.NE] 29 Mar 2019

StyleGAN

- Shaping the decoder input in style transfer is shaping the latent space for a GAN!
- Therefore, inserting “style” into a GAN Architecture is about understanding (and controlling) the image synthesis process in the generator of convolutional GANs.
- More specifically, aiming at improving
 - the distribution quality,
 - interpolation properties,
 - and latent space entanglement.

Abstract

We propose an alternative generator architecture for generative adversarial networks, borrowing from style transfer literature. The new architecture leads to an automatically learned, unsupervised separation of high-level attributes (e.g., pose and identity when trained on human faces) and stochastic variation in the generated images (e.g., freckles, hair), and it enables intuitive, scale-specific control of the synthesis. The new generator improves the state-of-the-art in terms of traditional distribution quality metrics, leads to demonstrably better interpolation properties, and also better disentangles the latent factors of variation. To quantify interpolation quality and disentanglement, we propose two new, automated methods that are applicable to any generator architecture. Finally, we introduce a new, highly varied and high-quality dataset of human faces.

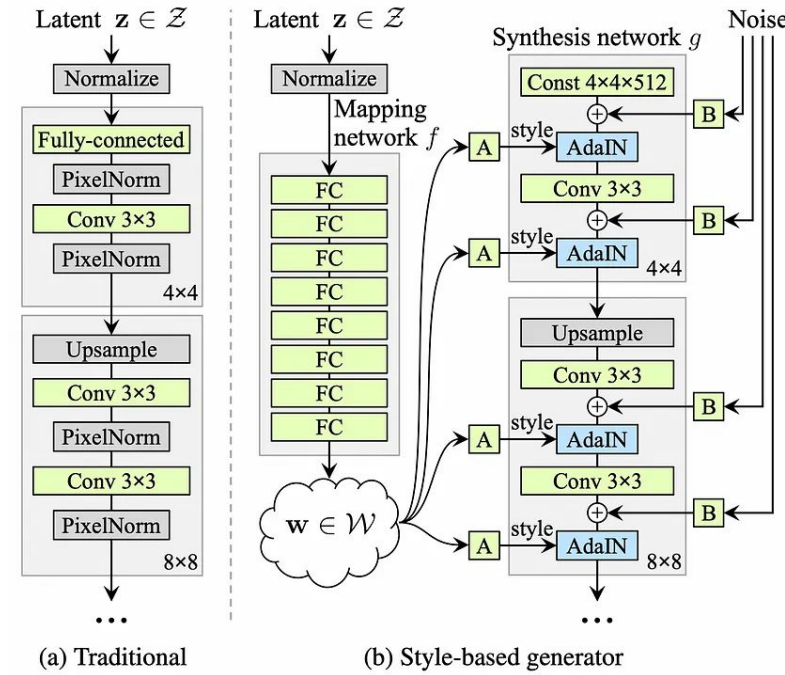
(e.g., pose, identity) from stochastic variation (e.g., freckles, hair) in the generated images, and enables intuitive scale-specific mixing and interpolation operations. We do not modify the discriminator or the loss function in any way, and our work is thus orthogonal to the ongoing discussion about GAN loss functions, regularization, and hyperparameters [24, 45, 5, 40, 44, 36].

Our generator embeds the input latent code into an intermediate latent space, which has a profound effect on how the factors of variation are represented in the network. The input latent space must follow the probability density of the training data, and we argue that this leads to some degree of unavoidable entanglement. Our intermediate latent space is free from that restriction and is therefore allowed to be disentangled. As previous methods for estimating the degree of latent space disentanglement are not directly applicable in our case, we propose two new automated metrics—

3v3 [cs.NE] 29 Mar 2019

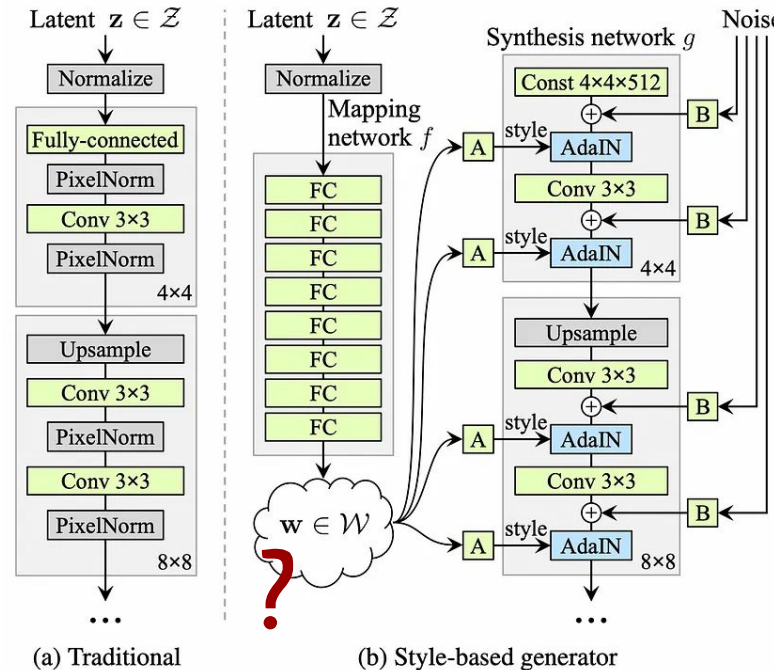
StyleGAN

- Instead of directly providing the latent code (Z) to the generator, a feed-forward network project (and disentangle) the input into an intermediate latent space (W).
- An affine transformation (i.e. norm.) can be computed from W , to directly control the adaptive instance normalization (AdaIN) after each convolution.
- Additional Gaussian noise is added to each feature map to effectively control stochastic variation without affecting the global attributes of the generated images.



StyleGAN

Our generator embeds the input latent code into an intermediate latent space, which has a profound effect on how the factors of variation are represented in the network. **The input latent space must follow the probability density of the training data, and we argue that this leads to some degree of unavoidable entanglement. Our intermediate latent space is free from that restriction and is therefore allowed to be disentangled.** As previous methods for estimating the degree of latent space disentanglement are not directly applicable in our case, we propose two new automated metrics — perceptual path length and linear separability — for quantifying these aspects of the generator. Using these metrics, we show that compared to a traditional generator architecture, our generator admits a more linear, less entangled representation of different factors of variation.

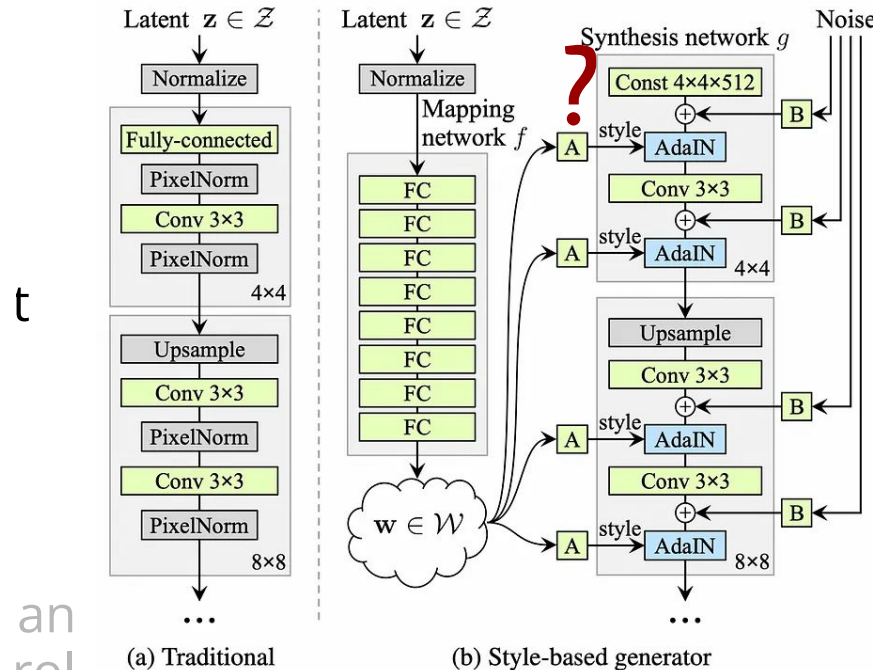


can control (AdaIN) after each convolution. o each feature map to effectively affecting the global attributes of

StyleGAN

2. Style-based generator

Traditionally the latent code is provided to the generator through an input layer, i.e., the first layer of a feed-forward network (Figure 1a). We depart from this design by omitting the input layer altogether and starting from a learned constant instead (Figure 1b, right). Given a latent code $\mathbf{z}$ in the input latent space $\mathcal{Z}$, a non-linear mapping network $f: \mathcal{Z} \rightarrow \mathcal{W}$ first produces $\mathbf{w} \in \mathcal{W}$ (Figure 1b, left). For simplicity, we set the dimensionality of both



anrol
AdaIN) after each convolution.

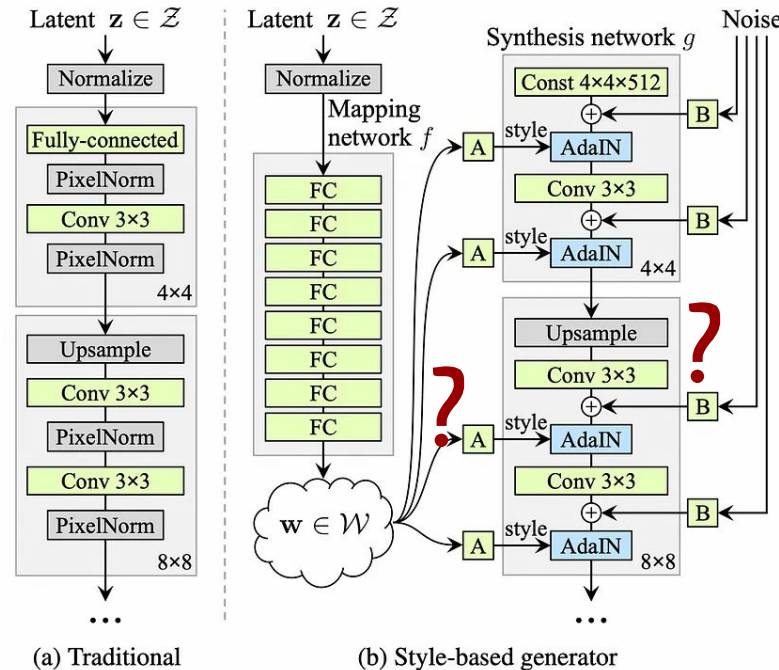
¹<https://github.com/NVlabs/stylegan>

- Additional Gaussian noise is added to each feature map to effectively control stochastic variation without affecting the global attributes of the generated images.

StyleGAN

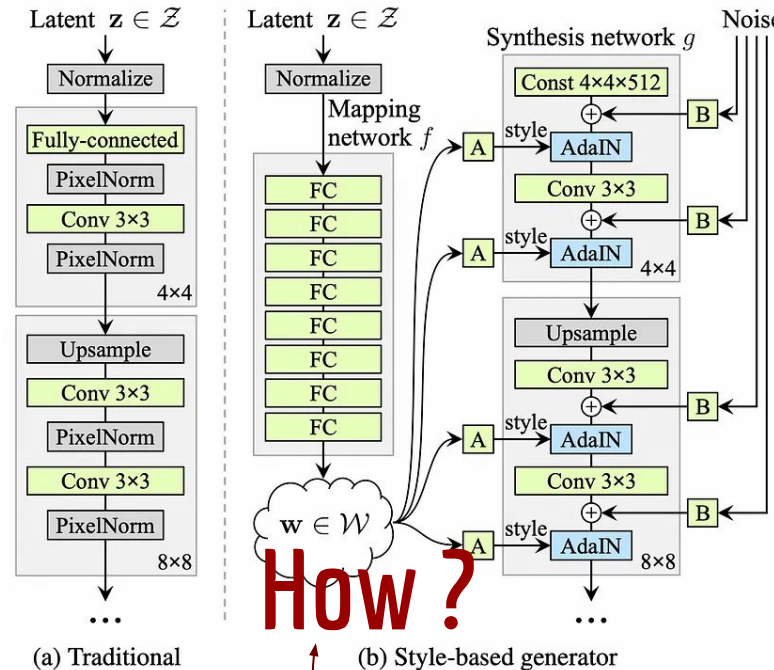
Figure 1. While a traditional generator [30] feeds the latent code though the input layer only, we first map the input to an intermediate latent space $\mathcal{W}$, which then controls the generator through adaptive instance normalization (AdaIN) at each convolution layer. Gaussian noise is added after each convolution, before evaluating the nonlinearity. Here “A” stands for a learned affine transform, and “B” applies learned per-channel scaling factors to the noise input. The mapping network f consists of 8 layers and the synthesis network g consists of 18 layers—two for each resolution ($4^2 - 1024^2$). The output of the last layer is converted to RGB using a separate 1×1 convolution, similar to Karras et al. [30]. Our generator has a total of 26.2M trainable parameters, compared to 23.1M in the traditional generator.

control stochastic variation without affecting the global attributes of the generated images.



StyleGAN

Our generator embeds the input latent code into an intermediate latent space, which has a profound effect on how the factors of variation are represented in the network. **The input latent space must follow the probability density of the training data, and we argue that this leads to some degree of unavoidable entanglement. Our intermediate latent space is free from that restriction and is therefore allowed to be disentangled.** As previous methods for estimating the degree of latent space disentanglement are not directly applicable in our case, we propose two new automated metrics — perceptual path length and linear separability — for quantifying these aspects of the generator. Using these metrics, we show that compared to a traditional generator architecture, our generator admits a more linear, less entangled representation of different factors of variation.

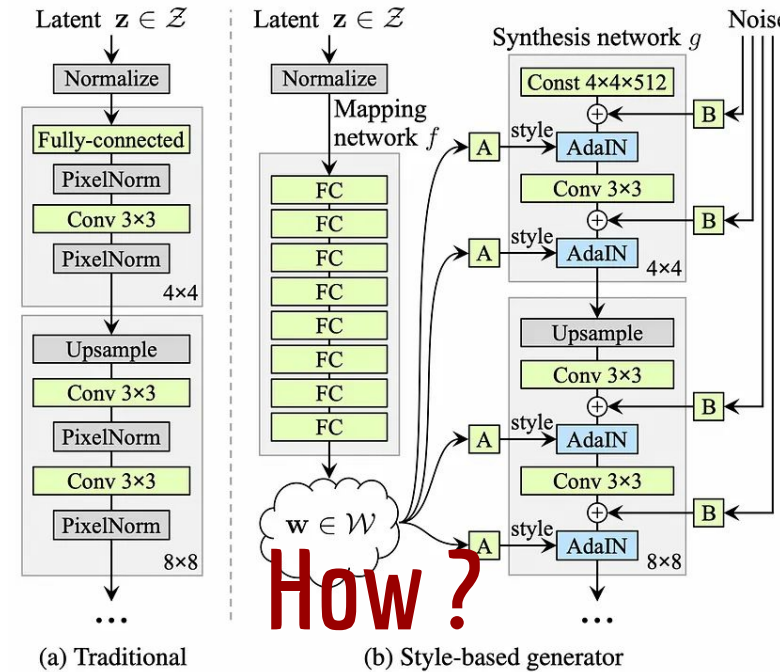


How?

can control (AdaIN) after each convolution. so each feature map to effectively affecting the global attributes of

StyleGAN

- the intuition behind the intermediate latent variable w being more disentangled compared to the original latent variable z lies in training strategy (i.e. loss function)



- During training, StyleGAN uses a technique called **path length regularization**. This regularization penalizes large changes in the intermediate latent space w , promoting a smoother and more linear interpolation between generated images.

Disentanglement

- *"There are various definitions for disentanglement [54, 50, 2, 7, 19], but a common goal is a **latent space that consists of linear subspaces, each of which controls one factor of variation.**"*
- *"A major benefit of our generator architecture is that the intermediate latent space W does not have to support sampling according to any **fixed** distribution (but instead $f(z)=w$)"*
- *"We **posit (assume)** that there is pressure for the generator to do so, as it should be easier to generate realistic images based on a disentangled representation than based on an entangled representation. As such, we expect the training to yield a less entangled W in an unsupervised setting."*

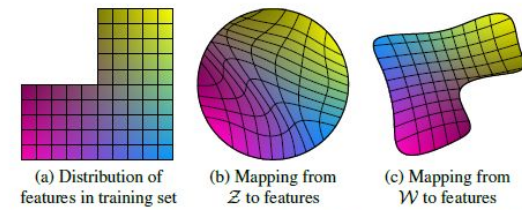
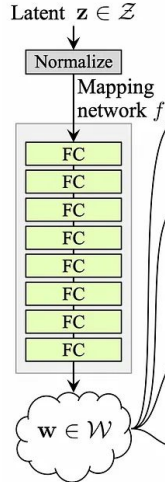


Figure 6. Illustrative example with two factors of variation (image features, e.g., masculinity and hair length). (a) An example training set where some combination (e.g., long haired males) is missing. (b) This forces the mapping from Z to image features to become curved so that the forbidden combination disappears in Z to prevent the sampling of invalid combinations. (c) The learned mapping from Z to W is able to “undo” much of the warping.



Path Length Regularization

- *"As noted by Laine [37], interpolation of latent-space vectors may yield surprisingly non-linear changes in the image. For example, features that are absent in either endpoint may appear in the middle of a linear interpolation path."*
- StyleGAN measures (using VGG16 embeddings) how drastic changes the image undergoes as they perform interpolation in the latent space.
- PLR is used to encourage a smoother and more predictable learning behavior in the latent space of the generator.
- It does this by penalizing large deviations in the latent space.

Path Length Regularization

- *"As noted by Laine [37], interpolation of latent-space yield surprisingly non-linear changes in the image features that are absent in either endpoint may occur along a linear interpolation path."*
- StyleGAN measures (using VGG16 embeddings) how drastic changes the image undergoes as they perform interpolation in the latent space.
- PLR is used to encourage a smoother and more predictable learning behavior in the latent space of the generator.
- It does this by penalizing large deviations in the latent space.

Method	FID	Path length		Separability
		full	end	
B Traditional 0 $\mathcal{Z}$	5.25	412.0	415.3	10.78
Traditional 8 $\mathcal{Z}$	4.87	896.2	902.0	170.29
Traditional 8 $\mathcal{W}$	4.87	324.5	212.2	6.52
Style-based 0 $\mathcal{Z}$	5.06	283.5	285.5	9.88
Style-based 1 $\mathcal{W}$	4.60	219.9	209.4	6.81
Style-based 2 $\mathcal{W}$	4.43	217.8	199.9	6.25
F Style-based 8 $\mathcal{W}$	4.40	234.0	195.9	3.79

Table 4. The effect of a mapping network in FFHQ. The number in method name indicates the depth of the mapping network. We see that FID, separability, and path length all benefit from having a mapping network, and this holds for both style-based and traditional generator architectures. Furthermore, a deeper mapping network generally performs better than a shallow one.



Path Length Regularization

The "full" column measures the total perceptual length of a segmented path between two points in the latent space, where the path is subdivided into linear segments and the perceptual difference between each segment is evaluated using an image distance metric. The average perceptual path length over all possible endpoints is then reported.

Interpolation of latent-space changes in the image as they move from one endpoint to another.

VGG16 embeddings as they change as they move from one endpoint to another.

- PLR is used to encourage a smoother learning behavior in the latent space
- It does this by penalizing large deviations

Method	FID	Path length		Separability
		full	end	
B Traditional 0 $\mathcal{Z}$	5.25	412.0	415.3	10.78
Traditional 8 $\mathcal{Z}$	4.87	896.2	902.0	170.29
Traditional 8 $\mathcal{W}$	4.87	324.5	212.2	6.52
Style-based 0 $\mathcal{Z}$	5.06	283.5	285.5	9.88
Style-based 1 $\mathcal{W}$	4.60	219.9	209.4	6.81
Style-based 2 $\mathcal{W}$	4.43	217.8	199.9	6.25
F Style-based 8 $\mathcal{W}$	4.40	234.0	195.9	3.79

Table 4. The effect of a mapping network in FFHQ. The number in method name indicates the depth of the mapping network. We see that FID, separability, and path length all benefit from having a mapping network, and this holds for both style-based and traditional generator architectures. Furthermore, a deeper mapping network tends to perform better than a shallower one.

The "end" column, on the other hand, measures the perceptual path length only between the two endpoints of the path, i.e., when the path is not subdivided into segments. This measurement is expected to be slightly biased in favor of the input latent space, as it does not take into account any potential regions in the intermediate latent space that are not on the input manifold and may be badly reconstructed by the generator.

Path Length Regularization

- *"As noted by Laine [37], interpolation of latent-space yield surprisingly non-linear changes in the image features that are absent in either endpoint may occur along a linear interpolation path."*
- StyleGAN measures (using VGG16 embeddings) changes the image undergoes as they perform the latent space.
- PLR is used to encourage a smoother and more predictable learning behavior in the latent space of the generator.
- It does this by penalizing large deviations in the latent space.

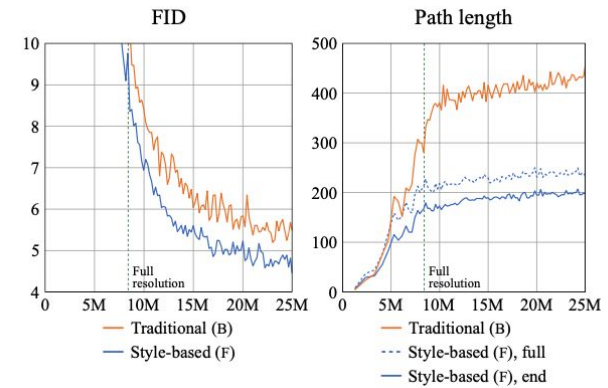


Figure 9. FID and perceptual path length metrics over the course of training in our configurations B and F using the FFHQ dataset. Horizontal axis denotes the number of training images seen by the discriminator. The dashed vertical line at 8.4M images marks the point when training has progressed to full 1024^2 resolution. On the right, we show only one curve for the traditional generator's path length measurements, because there is no discernible difference between full-path and endpoint sampling in $\mathcal{Z}$.

Test with noise

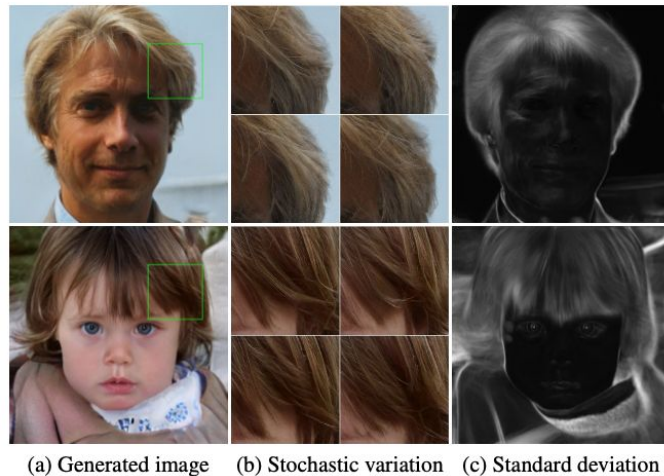


Figure 4. Examples of stochastic variation. (a) Two generated images. (b) Zoom-in with different realizations of input noise. While the overall appearance is almost identical, individual hairs are placed very differently. (c) Standard deviation of each pixel over 100 different realizations, highlighting which parts of the images are affected by the noise. The main areas are the hair, silhouettes, and parts of background, but there is also interesting stochastic variation in the eye reflections. Global aspects such as identity and pose are unaffected by stochastic variation.

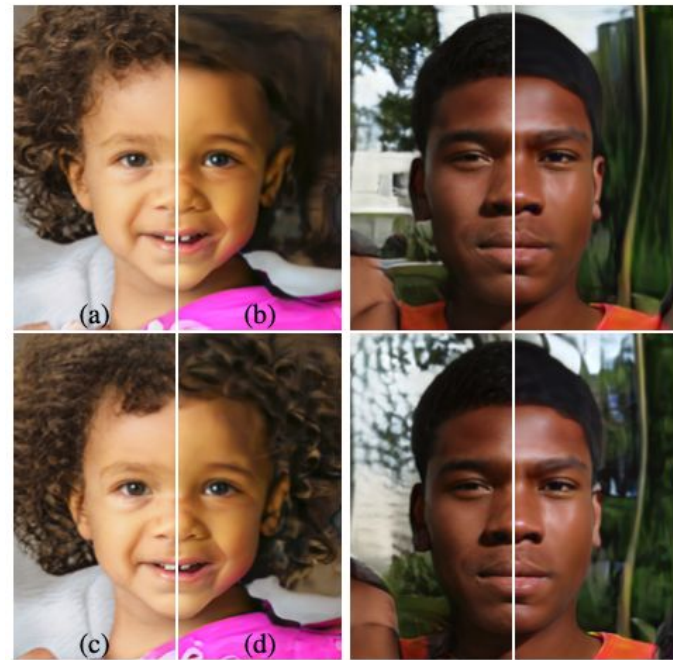


Figure 5. Effect of noise inputs at different layers of our generator. (a) Noise is applied to all layers. (b) No noise. (c) Noise in fine layers only ($64^2 - 1024^2$). (d) Noise in coarse layers only ($4^2 - 32^2$). We can see that the artificial omission of noise leads to featureless “painterly” look. Coarse noise causes large-scale curling of hair and appearance of larger background features, while the fine noise brings out the finer curls of hair, finer background detail, and skin pores.



StyleGAN 2

Abstract

additional random noise maps to the synthesis network. It has been demonstrated [24, 38] that this design allows the intermediate latent space $\mathcal{W}$ to be much less entangled than the input latent space $\mathcal{Z}$. In this paper, we focus all analysis solely on $\mathcal{W}$, as it is the relevant latent space from the synthesis network's point of view.

Many observers have noticed characteristic artifacts in images generated by StyleGAN [3]. We identify two causes for these artifacts, and describe changes in architecture and training methods that eliminate them. First, we investigate the origin of common blob-like artifacts, and find that the generator creates them to circumvent a design flaw in its architecture. In Section 2, we redesign the normalization used in the generator, which removes the artifacts. Second, we

The style-based GAN architecture (StyleGAN) yields state-of-the-art results in data-driven unconditional generative image modeling. We expose and analyze several of its characteristic artifacts, and propose changes in both model architecture and training methods to address them. In particular, we redesign the generator normalization, revisit progressive growing, and regularize the generator to encourage good conditioning in the mapping from latent codes to images. In addition to improving image quality, this path length regularizer yields the additional benefit that the generator becomes significantly easier to invert. This makes it possible to reliably attribute a generated image to a particular network. We furthermore visualize how well the generator utilizes its output resolution, and identify a capacity problem, motivating us to train larger models for additional quality improvements. Overall, our improved model redefines the state of the art in unconditional image modeling, both in terms of existing distribution quality metrics as well as perceived image quality.

- Same group (again NVIDIA author) came with the sequel the following year.
- StyleGAN2 is similar to its predecessor
- Its contributions are
 - First, adaptive instance normalization with a normalization
 - Secondly, an improved progressive growing is introduced
 - Additionally, new type of path length regularization

ements.

d replaced
ation.

ively

arization

for more on style GAN 2
[check this one](#) and [this one](#)



StyleGAN 2

Tero Karras
NVIDIASamuli Laine
NVIDIAMiika Aittala
NVIDIAJanne Hellsten
NVIDIAJaakko Lehtinen
NVIDIA and Aalto UniversityTimo Aila
NVIDIA

Abstract

The style-based GAN architecture (StyleGAN) yields state-of-the-art results in data-driven unconditional generative image modeling. We expose and analyze several of its characteristic artifacts, and propose changes in both model architecture and training methods to address them. In particular, we redesign the generator normalization, revisit progressive growing, and regularize the generator to encourage good conditioning in the mapping from latent codes to images. In addition to improving image quality, this path length regularizer yields the additional benefit that the generator becomes significantly easier to invert. This makes it possible to reliably attribute a generated image to

additional random noise maps to the synthesis network. It has been demonstrated [24, 38] that this design allows the intermediate latent space $\mathcal{W}$ to be much less entangled than the input latent space $\mathcal{Z}$. In this paper, we focus all analysis solely on $\mathcal{W}$, as it is the relevant latent space from the synthesis network's point of view.

Many observers have noticed characteristic artifacts in images generated by StyleGAN [3]. We identify two causes for these artifacts, and describe changes in architecture and training methods that eliminate them. First, we investigate the origin of common blob-like artifacts, and find that the generator creates them to circumvent a design flaw in its architecture. In Section 2, we redesign the normalization used in the generator, which removes the artifacts. Second, we

- Same group (again NVIDIA, the same first author) came with the second version, the following year.
- StyleGAN2 is similar to its predecessor, with some improvements.
- Its contributions are
 - First, adaptive instance normalization is redesigned and replaced with a normalization technique called weight demodulation.
 - Secondly, an improved training scheme upon progressively growing is introduced
 - Additionally, efficient regularization by “lazy regularization”

Droplet Artifacts

- Click [3] here

synthesis network's point of view.

Many observers have noticed characteristic artifacts in images generated by StyleGAN [3]. We identify two causes for these artifacts, and describe changes in architecture and



- "Even when the droplet may not be obvious in the final image, it is present in the intermediate feature maps of the generator."*
- "The anomaly starts to appear around 64×64 resolution, is present in all feature maps, and becomes progressively stronger at higher resolutions."*
- "The existence of such a consistent artifact is puzzling, as the discriminator (in the early progressive steps) should be able to detect it."*

Droplet Artifacts

- They pinpoint the problem to the AdaIN operation that normalizes the mean and variance of each feature map separately, thereby potentially destroying any information found in the magnitudes of the features relative to each other.
- They hypothesize that the droplet artifact is a result of the generator intentionally sneaking signal strength information past instance normalization: by creating a strong, localized spike that dominates the statistics, the generator can effectively scale the signal as it likes elsewhere.

Generator redesigned

- The new design removes the characteristic artifacts.
- While It retains full controllability.
- FID remains largely unaffected

Generator

- The new c
- While It re
- FID remai

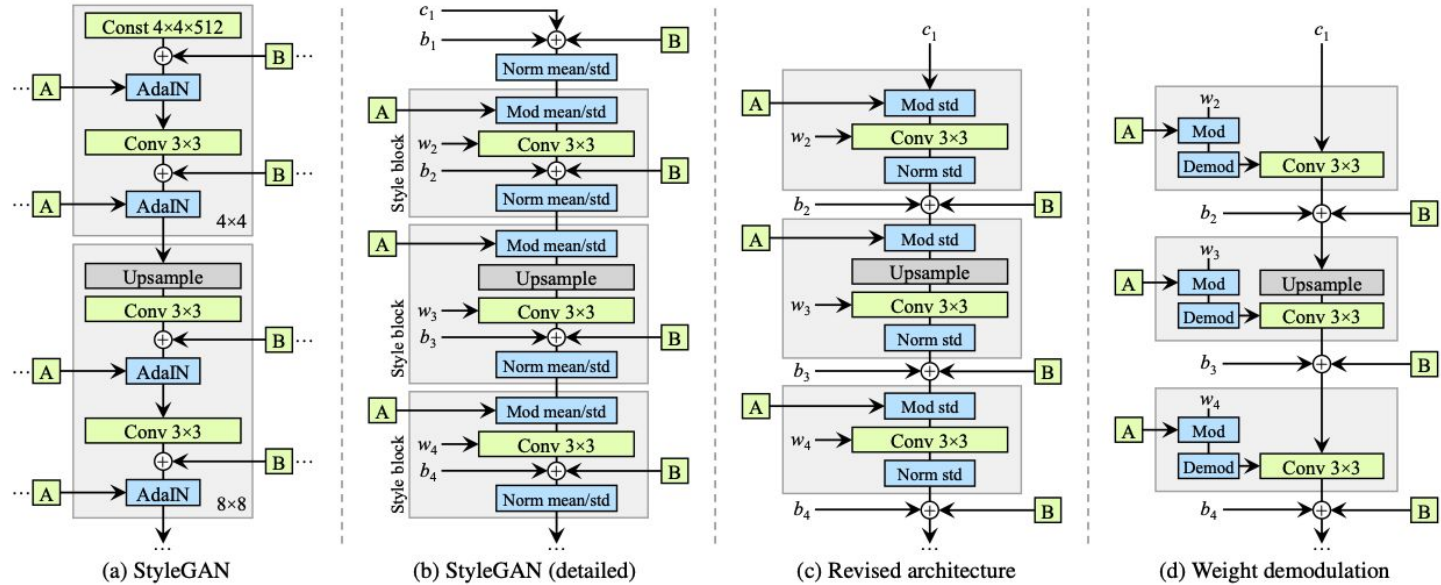


Figure 2. We redesign the architecture of the StyleGAN synthesis network. (a) The original StyleGAN, where $\boxed{A}$ denotes a learned affine transform from $\mathcal{W}$ that produces a style and $\boxed{B}$ is a noise broadcast operation. (b) The same diagram with full detail. Here we have broken the AdaIN to explicit normalization followed by modulation, both operating on the mean and standard deviation per feature map. We have also annotated the learned weights (w), biases (b), and constant input (c), and redrawn the gray boxes so that one style is active per box. The activation function (leaky ReLU) is always applied right after adding the bias. (c) We make several changes to the original architecture that are justified in the main text. We remove some redundant operations at the beginning, move the addition of b and $\boxed{B}$ to be outside active area of a style, and adjust only the standard deviation per feature map. (d) The revised architecture enables us to replace instance normalization with a “demodulation” operation, which we apply to the weights associated with each convolution layer.

Lazy Regularization

- Typically the main loss function and regularization terms are written as a single expression and are thus optimized simultaneously.
- The regularization terms can be computed less frequently than the main loss function, thus greatly diminishing their computational cost and the overall memory usage.
- Results show that no harm is caused when R1 regularization (penalizing the discriminator's gradients with respect to its input data) is performed only once every 16 minibatches.

Lazy Regularization

- Typically the main loss function and regularization terms are written as a single expression and are thus optimized simultaneously.
- The regularization terms can be computed less frequently than the main loss function, thus greatly diminishing their computational cost and the overall memory usage.
- Results show that no harm is caused when R1 regularization (penalizing the discriminator's gradients with respect to its input data) is performed only once every 16 minibatches.

Progressive Growing Revisited

- The authors also identify some other characteristic artifacts, such as a strong location preference for details and compromised shift invariance.
- In StyleGAN 2, the authors aim to address these artifacts by refining the progressive growing process to produce more spatially consistent and higher-quality images .
- The authors explore alternative network architectures to achieve high-quality image generation without relying on progressive growing. This involves re-evaluating the network design of StyleGAN and searching for an architecture that produces high-quality images without the need for the progressive growing technique.

What will we do next week?

- Next week: Disentanglement, Inversion, etc...